

DEVOPS MASTER COURSE

Volume 11A

Observability Foundations

Telemetry • Linux Monitoring • Metrics • Logs • Traces • SLIs/SLOs • Dashboards • Alerting

Chapters 1-10 | 2026 Edition

Production objective: build an observability foundation that can explain user impact, correlate system behavior across signals, detect data-quality failure, and support safe operational decisions - not merely display infrastructure graphs.

Beginner to Production-Ready DevOps and SRE Engineering

About This Part

Volume 11 begins the observability, monitoring, logging, tracing, and Site Reliability Engineering sequence. Part A establishes the concepts and operating contracts required before deep specialization in Prometheus, Grafana, log platforms, OpenTelemetry, tracing backends, profiling, alert engineering, incident response, and enterprise observability architecture.

The core principle is that observability starts with user journeys, operational questions, ownership, and trustworthy evidence. Tools are selected only after the required decisions, signals, context, reliability, security, privacy, retention, and cost boundaries are understood.

Foundation principle: a green dashboard is not proof of a healthy service, and missing telemetry must never be silently interpreted as success.

Volume 11 Roadmap

Part	Coverage	Status
11A	Observability foundations, telemetry, Linux monitoring, metrics, logs, traces, SLOs, dashboards and alerting	This book
11B	Prometheus architecture, exporters, service discovery, PromQL, recording rules, Alertmanager, HA and scale	Planned
11C	Grafana engineering, centralized logging, Loki, Elastic/OpenSearch patterns, pipelines, retention and governance	Planned
11D	OpenTelemetry, distributed tracing, Tempo/Jaeger patterns, APM, profiling and eBPF observability	Planned
11E	SRE operations, on-call, incidents, capacity, chaos, reliability, cost, security and governance	Planned
11F	Enterprise observability projects, troubleshooting, certifications, interviews and final capstone	Planned

Scope, Safety and Cost Controls

- Run load, failure, outage, network, storage, alert-routing, and data-loss tests only in disposable or explicitly approved nonproduction environments.
- Use synthetic services, identities, data, logs, traces, and secrets. Never place production credentials, customer data, internal hostnames, or employer configurations in course artifacts.
- Apply storage quotas, retention, ingestion limits, sampling, budgets, and cleanup before generating high-volume telemetry. Observability systems can consume substantial compute, memory, disk, network, and managed-service spend.
- Protect audit, security, trace, and log records according to their data classification. Telemetry can contain personal data, tokens, paths, queries, payload fragments, and infrastructure identifiers.
- Verify current product versions, supported configuration, image names, ports, licensing, feature status, and official guidance before production deployment.

How to Use This Book

1. Read the operational question and architecture notes before running commands.
2. Build each lab in a disposable environment and capture timestamps, configuration, output, screenshots, and cleanup evidence.
3. Generate known-good telemetry before introducing a controlled failure.
4. Validate normal, missing-data, delayed-data, duplicate-data, overload, failure, and recovery paths.
5. Explain each result in your own words and convert the lab into a sanitized portfolio artifact.

Part A Learning Outcomes

- Design an observability operating model with named users, questions, owners, service levels, trust boundaries, retention, and cost controls.
- Correlate metrics, logs, traces, events, profiles, resource context, release context, and platform self-telemetry.
- Troubleshoot Linux host and process behavior using CPU, memory, storage, network, service, and kernel evidence.
- Design metric and log schemas that aggregate correctly, avoid unbounded cardinality, and protect sensitive information.
- Define user-centric SLIs, SLOs, error budgets, dashboards, alerts, and runbooks.
- Build and validate a small observability platform project with controlled failure and recovery exercises.

Table of Contents

- Chapter 1 Observability, Monitoring and the SRE Operating Model
- Chapter 2 Telemetry Signals and End-to-End Data Flow
- Chapter 3 Service Inventory, Ownership and Observability Requirements
- Chapter 4 Linux Host and Process Monitoring Fundamentals
- Chapter 5 Metrics and Time-Series Data Fundamentals
- Chapter 6 Logs and Structured Event Engineering
- Chapter 7 Distributed Tracing and Context Propagation Fundamentals
- Chapter 8 SLIs, SLOs, Error Budgets and User-Centric Measurement
- Chapter 9 Dashboards, Alerting and Diagnostic Workflow Foundations
- Chapter 10 Foundation Observability Platform Project
- Part A Command and Configuration Reference
- Observability Foundation Completion Checklist
- Ten Additional Practice Projects
- 30-Day Foundation Practice Plan
- Glossary
- Official Documentation Index

Chapter 1: Observability, Monitoring and the SRE Operating Model

Understanding what must be known before choosing tools

Why this chapter matters: Teams often install dashboards before defining the decisions, users, ownership, and failure scenarios the observability system must support. This creates telemetry without operational clarity.

Learning Objectives

- Distinguish monitoring, observability, telemetry, diagnostics, audit evidence, and business analytics.
- Explain how observability supports development, operations, security, reliability, and product decisions.
- Define ownership, consumers, service levels, retention, and failure boundaries for an observability platform.
- Use hypothesis-driven investigation instead of dashboard guessing.
- Create a measurable foundation roadmap for a small service.

Core Concepts

Monitoring and observability

Monitoring evaluates known conditions and objectives through selected measurements. Observability is the capability to infer internal system behavior from externally available outputs, especially when the exact failure mode was not predicted. Mature operations use both: monitoring for explicit decisions and observability for explanation.

Telemetry and evidence

Telemetry includes metrics, logs, traces, profiles, events, and contextual metadata. Evidence is telemetry that has sufficient identity, time, integrity, retention, and interpretation to support a decision, incident timeline, audit, or change review.

Consumers and questions

Developers need code-level failure context; operators need service health and recovery signals; security teams need identity and threat evidence; product teams need user-impact and business-flow measurements. One dataset may serve several consumers, but each question requires an explicit contract.

SRE operating model

Site Reliability Engineering applies software engineering to operations. Reliability work is guided by service objectives, risk, error budgets, automation, incident learning, capacity, and reduction of repetitive toil rather than by an unlimited promise of uptime.

Architecture and Operating Decisions

- Start with user journeys and operational decisions, not with dashboard products.
- Assign owners for instrumentation, collection, storage, visualization, alerting, retention, privacy, and incident use.
- Separate service telemetry from audit and security records when access, immutability, or retention requirements differ.
- Define what happens when the monitoring system itself is degraded or unavailable.
- Review usefulness through incident outcomes, alert quality, investigation time, adoption, and cost.

Reference Configuration or Command Pattern

```
# Foundation discovery worksheet
service: checkout-api
owners:
  product: commerce
  engineering: checkout-platform
  operations: sre-oncall
user_journeys:
  - submit-order
  - retrieve-order-status
decisions:
  - is the service meeting its latency and success objectives?
  - is a new release harming customers?
  - which dependency is responsible for an error increase?
required_signals: [metrics, structured-logs, traces]
monitoring_rto_minutes: 30
telemetry_retention_days: 30
review_interval: quarterly
```

Interpretation: This is an operating contract, not a tool configuration. It creates the questions that later dashboards, alerts, logs, and traces must answer.

Production Readiness Checklist

- ☐ Critical services and user journeys have named owners.
- ☐ Each telemetry signal has a purpose, producer, consumer, schema, retention period, and access policy.
- ☐ The platform has service objectives and a documented degraded-mode procedure.
- ☐ Investigation, alert, audit, privacy, and cost requirements are explicitly separated.
- ☐ A review process removes unused dashboards, noisy alerts, unowned data, and high-cost cardinality.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Many dashboards, slow incidents	Dashboards are organized by infrastructure rather than user-impact questions	Create service and journey views linked to drill-down evidence	Reduced time to identify the affected service and dependency
Teams do not trust telemetry	Data ownership, timestamps, labels, or sampling are unclear	Publish signal contracts and validation checks	Known-good test event appears consistently across the pipeline
Observability fails during outage	Collection and storage share the same failure domain	Add buffering, isolation, external probes, and recovery procedures	Game day preserves or restores minimum visibility
Cost grows without value	High-cardinality or duplicate data has no consumer	Measure usage, apply budgets, aggregation, sampling, and expiry	Lower ingest/storage cost with unchanged required decisions

Hands-on Lab 1: Create an Observability Operating Charter

Prerequisites

- A sample service or imagined production system.
- A repository for diagrams, decisions, runbooks, and evidence.
- At least three operational scenarios: release regression, dependency failure, and capacity saturation.

Implementation

1. Identify the top three user journeys and the business consequence of failure.
2. List service components, dependencies, owners, environments, trust boundaries, and existing telemetry.
3. Write ten questions that engineers must answer during normal operation and incidents.
4. Map each question to metrics, logs, traces, profiles, synthetic checks, or business events.
5. Define platform ownership, service objectives, retention, access, privacy, cost, and degraded-mode behavior.
6. Run a tabletop exercise in which the primary dashboard system is unavailable during an application incident.

7. Publish the charter, open gaps as tracked work, and schedule the first usefulness review.

Validation Evidence

- A service map and consumer-question matrix are version controlled.
- Every critical question has an owner and at least one proposed evidence source.
- The degraded-mode exercise produces a recovery or alternative-observation procedure.

Cleanup

- Remove any real production identifiers or customer data from the learning repository.
- Close temporary accounts, tokens, dashboards, and test data created for the exercise.

Interview Questions and Model Answers

1. What is the practical difference between monitoring and observability?

Monitoring evaluates defined conditions and objectives; observability helps explain internal behavior from system outputs, including failures that were not fully anticipated.

2. Why is telemetry not automatically evidence?

Without reliable time, identity, context, integrity, retention, and interpretation, telemetry may be incomplete or misleading for a decision.

3. Who owns observability?

The platform team may own shared capabilities, but service teams must own instrumentation and the meaning of service-specific signals. Ownership must be explicit across the full pipeline.

4. How do you measure an observability platform?

Use signal availability, freshness, query performance, alert quality, investigation time, incident outcomes, adoption, coverage, and cost rather than counting dashboards.

Chapter Summary

- Observability begins with questions, ownership, and user impact.
- Monitoring and observability are complementary capabilities.
- Telemetry requires context and validation before it becomes evidence.
- The observability platform is itself a production service with failure modes and service objectives.

Chapter 2: Telemetry Signals and End-to-End Data Flow

Designing metrics, logs, traces, profiles, events, and context as one system

Why this chapter matters: Individual signals are useful, but incidents are resolved faster when engineers can move from a user symptom to a service metric, correlated trace, structured log, and resource context without losing identity or time.

Learning Objectives

- Explain the strengths and limitations of metrics, logs, traces, profiles, and events.
- Design producer, receiver, processor, exporter, storage, query, and visualization stages.
- Use consistent resource and service attributes to correlate signals.
- Recognize data-loss, duplication, delay, ordering, sampling, and schema risks.
- Design a vendor-neutral collection tier using OpenTelemetry concepts.

Core Concepts

Metrics

Metrics are numerical measurements aggregated over time. They are efficient for trends, objectives, capacity, and alerting but usually omit individual-event detail. Counters, gauges, histograms, and summaries have different semantics and aggregation behavior.

Logs and events

Logs record events with timestamps, severity, body, resource context, and attributes. Structured logs improve parsing and correlation. Audit events require stronger integrity and access controls than ordinary diagnostic logs.

Traces and profiles

A trace represents a request or workflow as spans connected across services. Profiles attribute resource consumption to code paths over time. Traces explain request flow; profiles explain where CPU, memory, or other resources are consumed.

Pipeline and context

A telemetry pipeline receives, enriches, filters, batches, samples, buffers, routes, and exports data. Resource attributes, service identity, environment, region, version, trace identifiers, and synchronized time make cross-signal analysis possible.

Architecture and Operating Decisions

- Use a stable service naming and resource-attribute convention across every signal.
- Collect close to the source when local buffering, enrichment, or network isolation is required.
- Centralize processing only when the capacity and failure-domain trade-off is acceptable.
- Treat sampling and filtering as governed data-loss decisions with observable rates.
- Define behavior for backpressure, exporter failure, queue exhaustion, and partial backend outage.

Reference Configuration or Command Pattern

```

receivers:
  otlp:
    protocols:
      grpc: {}
      http: {}
processors:
  memory_limiter:
    check_interval: 1s
    limit_mib: 256
  resource:
    attributes:
      - key: deployment.environment.name
        value: lab
        action: upsert
  batch: {}
exporters:
  debug:
    verbosity: normal
service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, resource, batch]
      exporters: [debug]
    metrics:
      receivers: [otlp]
      processors: [memory_limiter, resource, batch]
      exporters: [debug]
    logs:
      receivers: [otlp]
      processors: [memory_limiter, resource, batch]
      exporters: [debug]

```

Interpretation: An OpenTelemetry Collector pipeline is assembled from receivers, processors, exporters, and enabled service pipelines. Production configurations also require authentication, TLS, queueing, retry, telemetry, and capacity tests.

Production Readiness Checklist

- ☐ Service, namespace, environment, region, instance, and version attributes are consistent.
- ☐ Clock synchronization and timestamp semantics are documented.
- ☐ Queue, retry, backpressure, drop, sample, and export failure metrics are monitored.
- ☐ Sensitive fields are classified and transformed or removed before export.
- ☐ A known test request can be followed across required signals and storage backends.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Metrics arrive but traces do not	Trace pipeline is not enabled or exporter endpoint differs	Validate active pipelines, ports, protocol, TLS, and exporter status	Known test trace reaches backend with expected attributes
Duplicate telemetry	Two agents or retry paths collect the same source	Define one authoritative collection path and deduplication policy	Duplicate rate returns to expected baseline
Logs cannot link to traces	Trace and span identifiers are absent or formatted inconsistently	Inject context and use a compatible structured schema	Dashboard link opens the exact trace from a log record
Collector memory spikes	Batching, queue, cardinality, or exporter backpressure is unbounded	Apply limits, capacity, queues, sampling, and scaling	Load test remains within resource and drop objectives

Hands-on Lab 2: Build and Validate a Local Telemetry Pipeline

Prerequisites

- Docker or a local OpenTelemetry Collector binary.
- A sample application or telemetry generator.
- A repository for collector configuration and test evidence.

Implementation

1. Create a Collector configuration with OTLP receivers, memory limiter, resource enrichment, batching, and debug exporters.
2. Start the Collector and inspect its own startup and health telemetry.
3. Send one trace, one metric, and one log record from a sample application or telemetry generator.
4. Verify service name, environment, version, timestamps, trace ID, and span ID.
5. Stop the exporter destination or use an invalid endpoint to observe retry, queue, and failure behavior.
6. Restore the destination and determine whether queued data is delivered, dropped, or duplicated.
7. Document the data flow, ports, trust boundaries, failure modes, and production improvements.

Validation Evidence

- Collector configuration is version controlled and syntactically validated.
- Known telemetry records appear with correct resource context.
- Failure and recovery behavior is captured with collector self-telemetry.

Cleanup

- Stop and remove containers, volumes, generated data, and temporary certificates.
- Delete test attributes that contain local usernames, hostnames, or sensitive paths.

Interview Questions and Model Answers

1. When should you use metrics instead of logs?

Use metrics for efficient trends, objectives, aggregation, capacity, and alert conditions; use logs when event detail and contextual fields are needed.

2. What is context propagation?

It carries identifiers and metadata across process and service boundaries so spans, logs, and related operations can be correlated.

3. Why monitor the telemetry pipeline itself?

A silent collection failure can make a healthy-looking dashboard false. Pipeline freshness, drops, queues, retries, errors, and capacity are part of system correctness.

4. What is the risk of centralizing all telemetry processing?

A central tier simplifies governance but can become a bottleneck, broad failure domain, sensitive-data concentration point, and high-cost dependency.

Chapter Summary

- Signals answer different questions and should be correlated through stable context.
- The collection pipeline can lose, delay, duplicate, or transform data.
- Sampling and filtering are explicit risk and cost decisions.
- Collector health and data quality must be observable.

Chapter 3: Service Inventory, Ownership and Observability Requirements

Turning architecture and user journeys into measurable contracts

Why this chapter matters: Observability coverage becomes unreliable when teams do not know which services exist, who owns them, which dependencies are critical, and what each environment is expected to expose.

Learning Objectives

- Build a service catalog that links ownership, repositories, runtimes, dependencies, objectives, and telemetry.
- Classify services by criticality, data sensitivity, change rate, and support model.
- Define minimum observability requirements as a versioned platform contract.
- Identify blind spots, orphaned services, and unsupported telemetry paths.
- Use onboarding and decommissioning workflows to keep inventory current.

Core Concepts

Service inventory

A service inventory is an authoritative record of deployable and operational units, their owners, repositories, runtime locations, dependencies, user journeys, data classifications, and lifecycle states.

Criticality and tiers

Tiering should reflect business impact, recovery needs, dependency centrality, regulatory obligations, and support model. Higher tiers require stronger objectives, synthetic checks, retention, alert coverage, recovery, and review.

Minimum telemetry contract

A contract defines required service identity, request measurements, dependency measurements, structured logs, trace propagation, release annotations, dashboards, alerts, runbooks, and data-quality checks.

Lifecycle governance

New services should satisfy onboarding gates before production. Retired services must remove alerts, dashboards, credentials, collectors, storage, and ownership records to avoid stale cost and confusion.

Architecture and Operating Decisions

- Choose an authoritative catalog and automate updates from source, deployment, and runtime evidence.
- Require owners and escalation paths before a service enters a supported production tier.
- Use platform defaults but allow documented exceptions with owner, risk, expiry, and review.
- Track telemetry coverage and data quality as release-readiness signals.
- Make decommissioning a first-class workflow rather than simply stopping workloads.

Reference Configuration or Command Pattern

```
apiVersion: platform.example/v1
kind: ServiceRecord
metadata:
  name: payments-api
spec:
  tier: 1
  owner: payments-platform
  repository: org/payments-api
  environments: [staging, production]
  dependencies: [identity-api, payment-gateway]
  observability:
    metrics: required
    structuredLogs: required
    tracePropagation: required
    syntheticCheck: required
    sloDocument: docs/slo.yaml
    runbook: docs/runbook.md
  dataClassification: confidential
  lifecycle: active
```

Interpretation: The schema can live in a developer portal, repository, CMDB, or catalog API. The key requirement is authoritative ownership and automated validation, not a specific product.

Production Readiness Checklist

- ☐ Every production service has an active owner, escalation path, repository, and runtime location.
- ☐ Critical dependencies and user journeys are recorded.
- ☐ Service tier determines required telemetry, objectives, retention, and support.
- ☐ Exceptions have risk, owner, expiry, and compensating controls.
- ☐ Decommissioned services remove monitoring objects, credentials, storage, and catalog entries.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Alert has no responder	Service ownership or escalation data is missing	Block supported production status until ownership exists	Test alert routes to an accountable team
Catalog says healthy, runtime absent	Inventory updates are manual or stale	Reconcile catalog with deployment and runtime evidence	Drift report detects removed or unexpected service
Critical dependency is invisible	Only direct application metrics are instrumented	Add dependency SLIs, traces, and external checks	Failure identifies dependency and user impact
Retired service still costs money	Dashboards, logs, indexes, or collectors were not decommissioned	Use lifecycle checklist and automated cleanup	No ingest, alerts, credentials, or storage remain

Hands-on Lab 3: Create a Service Catalog and Telemetry Contract

Prerequisites

- Three sample services with at least one dependency relationship.
- A Git repository or simple catalog file.
- An agreed set of service tiers.

Implementation

1. Define a service-record schema containing ownership, tier, repositories, environments, dependencies, data classification, objectives, dashboards, alerts, and runbooks.
2. Create records for three services and intentionally leave one ownership field incomplete.
3. Write validation that rejects missing required fields for production-tier services.
4. Map runtime evidence to inventory and identify orphaned or unexpected workloads.

5. Define a minimum observability contract for each tier.
6. Create an exception record with owner, reason, compensating control, expiry, and review date.
7. Run a mock decommissioning and verify removal of telemetry and access artifacts.

Validation Evidence

- Schema and service records are version controlled.
- Validation demonstrates both accepted and rejected records.
- A drift or orphan report and decommission checklist are produced.

Cleanup

- Remove test integrations and catalog entries.
- Delete temporary alert routes, dashboards, API keys, and generated service data.

Interview Questions and Model Answers

1. Why is a service catalog relevant to observability?

Telemetry without ownership and service context cannot reliably route alerts, define objectives, control retention, or support incident response.

2. How should service tiers be chosen?

Use business impact, user dependency, recovery requirements, data sensitivity, regulatory obligations, and support model rather than team preference alone.

3. What belongs in an observability contract?

Identity, required signals, schemas, objectives, release annotations, dashboards, alerts, runbooks, retention, access, quality checks, and exception handling.

4. How do you prevent a catalog from becoming stale?

Automate reconciliation from repositories, deployment systems, runtime inventory, and ownership directories, then alert on drift and missing owners.

Chapter Summary

- Inventory and ownership are prerequisites for reliable telemetry.
- Criticality should drive observability requirements.
- Platform contracts standardize minimum behavior without hiding service-specific meaning.
- Onboarding and decommissioning must update the full observability lifecycle.

Chapter 4: Linux Host and Process Monitoring Fundamentals

Reading CPU, memory, disk, network, process, and service behavior safely

Why this chapter matters: Application incidents often become confusing because engineers cannot distinguish CPU pressure from load average, memory use from memory pressure, filesystem capacity from I/O latency, or a listening socket from a reachable service.

Learning Objectives

- Inspect Linux load, CPU, memory, storage, network, processes, services, and logs.
- Interpret rates and trends instead of relying on one snapshot.
- Correlate application symptoms with operating-system evidence.
- Use /proc and standard utilities without causing additional load.
- Build a repeatable host investigation record.

Core Concepts

CPU and scheduler

CPU utilization shows time categories; load average represents runnable or uninterruptible work relative to available capacity. High load can result from CPU saturation or blocked I/O, so run queue, per-CPU activity, process state, and latency must be interpreted together.

Memory and pressure

Linux uses available memory for cache. Free memory alone is not a failure signal. Evaluate available memory, swap activity, reclaim, major faults, cgroup limits, out-of-memory events, and application working set.

Storage and network

Filesystem fullness is different from device latency and saturation. Network reachability is different from a local listening socket. Inspect capacity, inodes, device queues, error counters, retransmissions, routes, DNS, firewall, and application responses.

Process and service context

Processes run under users, cgroups, namespaces, limits, and service managers. A healthy process identifier does not prove the service is responsive, reachable, or meeting objectives.

Architecture and Operating Decisions

- Collect a lightweight baseline before an incident.
- Use rate-based tools over an interval and record timestamps.
- Begin with user symptom, then inspect service, dependencies, host pressure, and recent change.
- Avoid destructive commands and high-frequency diagnostics on production without approval.
- Link host telemetry to service, environment, instance, and deployment version.

Reference Configuration or Command Pattern

```
# Fast, low-risk host triage
date -Is
uptime
free -h
vmstat 1 5
pidstat -dur 1 5
iostat -xz 1 5
ss -s
ss -lntup
ip -s link
df -hT
df -ih
systemctl --failed
journalctl -p warning..alert --since "15 minutes ago" --no-pager

# Process context
ps -eo pid,ppid,user,stat,%cpu,%mem,etimes,cmd --sort=-%cpu | head
cat /proc/pressure/cpu
cat /proc/pressure/memory
cat /proc/pressure/io
```

Interpretation: Run commands with appropriate privilege and bounded duration. Tool availability varies by distribution; pidstat and iostat are commonly provided by the sysstat package.

Production Readiness Checklist

- ☐ Host clocks are synchronized and timestamps are captured.
- ☐ CPU, memory, disk, network, process, service, and kernel evidence is collected over an interval.
- ☐ Container or cgroup limits are checked where applicable.
- ☐ User-path and dependency tests accompany host-level evidence.
- ☐ Commands, access, and outputs are sanitized before external sharing.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
High load, moderate CPU	Tasks are blocked in uninterruptible I/O	Inspect process state, iostat latency, storage and kernel logs	Blocked tasks and device latency return to baseline
Low free memory	Page cache is being used normally	Check available memory, swap, faults, PSI and OOM history	No sustained pressure or swap storm is present
Port is listening, users fail	Routing, firewall, dependency, TLS, or application path is broken	Test from client path and inspect network/application evidence	External request succeeds with expected latency
Disk has space, writes stall	Device latency, inode exhaustion, quota, filesystem or remote storage issue	Inspect inodes, queues, errors, mounts and quotas	Write latency and error rate recover

Hands-on Lab 4: Build a Linux Host Baseline and Incident Snapshot

Prerequisites

- A disposable Linux VM or WSL2 environment.
- sudo access for approved package installation and service inspection.
- A small test service such as Python HTTP server or nginx.

Implementation

1. Record OS, kernel, CPU, memory, block devices, filesystems, interfaces, routes, DNS, and running services.
2. Install or verify procps, iproute2, sysstat, and curl tools.
3. Run a five-minute baseline of vmstat, pidstat, iostat, ss, filesystem, and service checks.
4. Generate controlled CPU load and compare utilization, run queue, load average, and process evidence.
5. Generate controlled disk activity and compare throughput, queue, utilization, and latency.

6. Stop the test service while leaving its host available; prove the difference between host reachability and service health.
7. Write a timestamped triage template and recovery summary.

Validation Evidence

- Baseline and controlled-failure outputs are preserved.
- The report distinguishes CPU saturation, I/O wait, memory pressure, and service failure.
- Recovery is independently verified through the user path.

Cleanup

- Stop load generators and remove temporary files and packages not required for future labs.
- Restore services, resource limits, and firewall changes.

Interview Questions and Model Answers

1. What does Linux load average measure?

It reflects the average number of tasks runnable or in uninterruptible sleep over time; it must be interpreted relative to CPU capacity and I/O evidence.

2. Why can free memory be low on a healthy server?

Linux uses memory for page cache and reclaims it when needed. Available memory, swap activity, faults, PSI, and OOM events are more useful than free memory alone.

3. How do you distinguish disk capacity from disk performance?

Capacity uses filesystem bytes and inodes; performance uses device throughput, queue depth, utilization, latency, errors, and application I/O behavior.

4. Does a listening port prove a service is healthy?

No. It proves a local socket exists. Reachability, protocol behavior, dependencies, latency, correctness, and user outcomes require separate validation.

Chapter Summary

- Linux diagnosis requires correlated rates, states, and timestamps.
- Load, utilization, pressure, capacity, and latency are different concepts.
- A running process or listening port does not prove service health.
- Safe baselines make incident evidence faster and more reliable.

Chapter 5: Metrics and Time-Series Data Fundamentals

Designing measurements that aggregate correctly and remain affordable

Why this chapter matters: Poor metric semantics and uncontrolled labels can create misleading dashboards, unusable alerts, and expensive time-series growth even when the monitoring software is operating correctly.

Learning Objectives

- Explain time series, samples, labels, dimensions, scrape intervals, and aggregation.
- Choose counters, gauges, histograms, and summaries correctly.
- Apply naming, units, reset handling, and label-cardinality controls.
- Recognize missing data, stale series, delayed samples, and aggregation errors.
- Create service metrics for traffic, errors, latency, and saturation.

Core Concepts

Time-series model

A time series is identified by a metric name and a set of dimensions or labels, with timestamped samples. Every unique label combination creates another series, so dimensions are both analytical power and resource cost.

Metric types

Counters increase except for reset and are analyzed as rates or increases. Gauges move up and down. Histograms count observations in configured buckets and support aggregation and quantiles. Summaries calculate client-side quantiles that are generally difficult to aggregate across instances.

Cardinality and identity

Unbounded values such as user ID, request ID, timestamp, email, or raw URL path should not become metric labels. Keep high-detail identity in logs or traces and use bounded dimensions in metrics.

Golden signals and RED/USE

Common service measurements include traffic, errors, latency, and saturation. RED emphasizes request rate, errors, and duration for services. USE emphasizes utilization, saturation, and errors for resources. These frameworks guide discovery but do not replace user-specific SLIs.

Architecture and Operating Decisions

- Define metric meaning, type, unit, labels, owner, and expected cardinality before release.
- Use base units and consistent naming conventions.
- Prefer histograms for aggregatable latency distributions and choose buckets from real objectives and traffic.
- Record deployment version and stable service context, but avoid per-request labels.
- Set cardinality budgets and monitor active series, churn, ingestion, and query cost.

Reference Configuration or Command Pattern

```
# Prometheus text exposition example
# HELP http_requests_total Total HTTP requests processed.
# TYPE http_requests_total counter
http_requests_total{service="catalog",method="GET",route="/items",status_code="200"} 1842

# HELP http_request_duration_seconds Request latency distribution.
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{service="catalog",route="/items",le="0.1"} 1600
http_request_duration_seconds_bucket{service="catalog",route="/items",le="0.3"} 1800
http_request_duration_seconds_bucket{service="catalog",route="/items",le="1"} 1838
http_request_duration_seconds_bucket{service="catalog",route="/items",le="+Inf"} 1842
http_request_duration_seconds_sum{service="catalog",route="/items"} 132.7
http_request_duration_seconds_count{service="catalog",route="/items"} 1842
```

Interpretation: The route label uses a normalized route template rather than a raw path containing item identifiers. Histogram bucket selection should reflect the service latency objective and observed distribution.

Production Readiness Checklist

- ☐ Every metric has documented type, unit, labels, owner, and intended decision.
- ☐ Labels are bounded and reviewed for privacy and cardinality.
- ☐ Counter reset, scrape failure, missing data, and stale-target behavior are understood.
- ☐ Histograms use buckets that support required objectives and aggregation.
- ☐ Metric pipeline capacity and series growth are monitored.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Request rate becomes negative	Raw counter differences ignore process reset	Use reset-aware rate or increase functions	Rate remains valid across restart test
Series count grows continuously	Unbounded label such as request ID or raw path	Remove or normalize label and expire old series	Active series and churn return within budget
Average latency looks healthy	Tail latency is hidden by averages	Use histogram distribution and objective-aligned quantiles	Slow-request population becomes visible
No data appears as zero	Query or dashboard masks missing targets	Represent missing data explicitly and alert on collection health	Stopped target produces missing-data state, not healthy zero

Hands-on Lab 5: Design and Expose a Small Service Metric Set

Prerequisites

- A sample HTTP service in any language.
- A local Prometheus-compatible scrape endpoint or text-file test.
- A workload generator such as curl loop or approved HTTP load tool.

Implementation

1. Define traffic, error, latency, and saturation questions for the service.
2. Create a counter for requests, a histogram for request duration, and a gauge for in-flight work.
3. Use bounded labels for method, normalized route, and status class.
4. Generate successful, slow, and failed requests and inspect the exposed samples.
5. Restart the application and verify counter-reset-aware analysis.
6. Introduce an intentionally unbounded label in a lab branch, measure the growth, then remove it.
7. Document metric contracts, expected series count, and objective-aligned histogram buckets.

Validation Evidence

- Metric exposition includes correct HELP and TYPE metadata.
- Tests demonstrate success, failure, latency, restart, and cardinality behavior.
- A metric contract records meaning, units, labels, owner, and cost expectation.

Cleanup

- Stop test workloads and remove intentionally unsafe label instrumentation.
- Delete temporary time-series storage or containers if not required.

Interview Questions and Model Answers

1. Why should counters usually be queried as rates?

The raw value is cumulative and resets on restart. Rate or increase functions express change over time and can account for resets.

2. What is metric cardinality?

It is the number of unique time series created by label combinations. High or unbounded cardinality increases memory, storage, and query cost.

3. When would you choose a histogram?

When you need an aggregatable distribution, objective-based bucket counts, or latency quantiles across multiple instances.

4. What is wrong with treating missing data as zero?

Missing telemetry may indicate a failed target or collection pipeline. Converting it to zero can make an outage appear healthy.

Chapter Summary

- Metric semantics determine whether aggregation is correct.
- Labels must be bounded, useful, and affordable.
- Histograms expose distributions that averages hide.
- Missing telemetry is a distinct operational state.

Chapter 6: Logs and Structured Event Engineering

Creating diagnostic records that are searchable, correlated, secure, and governable

Why this chapter matters: Logs become expensive noise when they lack stable fields, severity discipline, correlation identifiers, ownership, retention, and protection against secrets or personal data.

Learning Objectives

- Distinguish diagnostic logs, audit logs, security events, and business events.
- Create structured records with stable schemas and contextual fields.
- Use severity, message design, trace correlation, and error classification consistently.
- Design collection, parsing, buffering, indexing, retention, and access controls.
- Prevent secret leakage, injection, duplication, and unbounded event volume.

Core Concepts

Log classes

Diagnostic logs explain system behavior; audit logs record security-relevant actions; security events support detection; business events describe domain outcomes. Their integrity, access, retention, and immutability requirements can differ.

Structured records

A useful structured log contains event name, timestamp, severity, service/resource identity, environment, version, trace and span identifiers, outcome, error type, and bounded attributes. Human-readable messages remain helpful, but fields support reliable queries.

Severity and actionability

Severity should describe operational significance, not developer emotion. Repeated expected events should not be emitted as errors. Alerts should not be created simply from every ERROR record without user-impact logic.

Pipeline and governance

Agents or collectors read sources, parse and enrich records, buffer data, route by class, and export to storage. Controls include schema validation, redaction, quotas, retention, immutability, access review, and legal or privacy requirements.

Architecture and Operating Decisions

- Define event names and schemas as versioned interfaces.
- Keep secrets, credentials, payment data, and unnecessary personal data out of logs by design.
- Use trace and span identifiers to correlate records, but do not use them as metric labels.
- Route audit and security records to appropriate protected storage.
- Set volume budgets, sampling rules, retention, archive, and deletion by log class.

Reference Configuration or Command Pattern

```
{
  "timestamp": "2026-08-01T05:30:14.482Z",
  "severity": "ERROR",
  "event_name": "checkout.payment_authorization_failed",
  "service.name": "checkout-api",
  "deployment.environment.name": "lab",
  "service.version": "1.4.2",
  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
  "span_id": "00f067aa0ba902b7",
  "outcome": "failure",
  "error.type": "gateway_timeout",
  "dependency.name": "payment-gateway",
  "duration_ms": 3021,
  "message": "Payment authorization exceeded dependency timeout"
}
```

Interpretation: The record contains operational context without customer identity, card data, credentials, or raw payloads. Field names should be governed and consistent across services.

Production Readiness Checklist

- ☐ Log classes, schemas, owners, retention, and access policies are defined.
- ☐ Sensitive-data tests run before release and at collection boundaries.
- ☐ Trace correlation and service resource fields are present.
- ☐ Parser failures, dropped records, backlog, late arrival, and duplicate rates are monitored.
- ☐ High-volume or low-value events have limits, sampling, or aggregation.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Search cannot group failures	Messages vary and structured error fields are absent	Create stable event name, outcome and error type	Query groups equivalent failures across instances
Secrets appear in logs	Raw request, environment, or exception data is emitted	Redact at source, test, rotate exposed secret, and review access	Secret scanner and controlled test confirm removal
Log backlog grows	Exporter or storage is slow and queues are unbounded	Apply buffering limits, routing, retry, shedding and capacity	Backlog drains within objective without uncontrolled loss
Same event appears twice	Multiple agents collect the same file or retries lack identity	Define one source path and event identity/deduplication	Duplicate ratio returns to baseline

Hands-on Lab 6: Create, Validate and Govern Structured Logs

Prerequisites

- A sample application that can emit JSON.
- A local file, stdout collector, or log backend.
- A synthetic secret string for leakage testing; never use a real credential.

Implementation

1. Define schemas for one success event, one dependency failure, and one administrative audit event.
2. Emit JSON logs containing service identity, environment, version, trace context, outcome, duration, and error type.
3. Create a controlled exception and verify stack information is useful without exposing secrets.
4. Insert a synthetic secret into a request and verify automated detection or redaction.
5. Stop the log destination to observe buffering, retry, backlog, and loss behavior.
6. Restore the destination and verify ordering, duplicates, and late arrival.
7. Write retention and access policies for diagnostic versus audit records.

Validation Evidence

- Schema examples and validation results are version controlled.
- Synthetic secret tests prove prevention or redaction.
- Pipeline failure and recovery metrics show backlog and delivery outcome.

Cleanup

- Delete the synthetic secret and generated records from all local storage.
- Remove temporary collectors, credentials, indexes, and test access grants.

Interview Questions and Model Answers

1. Why prefer structured logs?

Stable fields allow reliable filtering, aggregation, correlation, routing, and schema validation while retaining a readable message.

2. Should every error log create a page?

No. Paging should be tied to actionable user impact or objective risk. Logs provide evidence; alert policy determines urgency.

3. How do audit logs differ from diagnostic logs?

Audit logs often require stronger completeness, integrity, restricted access, retention, and immutability because they prove security-relevant actions.

4. How do you reduce log cost safely?

Remove low-value events, aggregate repeated information, use class-based retention, sample only where acceptable, compress, tier storage, and measure consumer usage.

Chapter Summary

- Logs need stable schemas, context, and governance.
- Diagnostic, audit, security, and business records have different requirements.
- Sensitive data should be prevented at source and controlled throughout the pipeline.
- Pipeline backlog, loss, parsing, duplication, and cost must be observable.

Chapter 7: Distributed Tracing and Context Propagation Fundamentals

Following a request across services, queues, databases, and asynchronous work

Why this chapter matters: In distributed systems, a user-visible delay or failure may cross many services and asynchronous boundaries. Without trace context and meaningful spans, teams correlate timestamps manually and often blame the wrong component.

Learning Objectives

- Explain traces, spans, parent-child relationships, links, attributes, events, status, and resources.
- Propagate context across HTTP, messaging, and asynchronous work.
- Choose span boundaries and attributes that answer dependency and latency questions.
- Understand head, tail, and probabilistic sampling trade-offs.
- Correlate traces with metrics, logs, and releases.

Core Concepts

Trace model

A trace represents an end-to-end operation. Spans represent timed operations with parent relationships, attributes, events, status, and resource context. A trace is useful only when span names and attributes describe actual service and dependency behavior.

Propagation

Trace context is carried across process boundaries using protocol headers or message metadata. Missing or overwritten context fragments a request into unrelated traces. Links can connect asynchronous or batch relationships that are not strict parent-child operations.

Instrumentation

Automatic instrumentation provides broad coverage quickly. Manual instrumentation adds domain operations, important attributes, and meaningful events. Duplicate instrumentation, excessive spans, sensitive attributes, and unstable names must be controlled.

Sampling

Head sampling decides early and is simple but may discard rare failures. Tail sampling evaluates more complete traces but requires collector state, latency, and capacity. Sampling policy must be observable and aligned with incident and cost needs.

Architecture and Operating Decisions

- Use stable span names based on operations or route templates, not raw identifiers.
- Propagate context through every supported protocol and asynchronous boundary.
- Record errors and dependency attributes without placing sensitive payloads into spans.
- Preserve representative baseline traffic while prioritizing rare errors or slow traces within budget.
- Link traces to service metrics, logs, deployment version, and incident timelines.

Reference Configuration or Command Pattern

```
# W3C Trace Context example
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01

# Conceptual span attributes
name: "POST /orders"
kind: SERVER
attributes:
  service.name: checkout-api
  http.request.method: POST
  http.route: /orders
  http.response.status_code: 503
  server.address: checkout.internal
  error.type: dependency_timeout
  deployment.environment.name: lab
  service.version: 1.4.2
```

Interpretation: Do not create span names or attributes from raw customer identifiers or unbounded URLs. Use semantic conventions supported by the instrumentation ecosystem where appropriate.

Production Readiness Checklist

- ☐ Trace context propagates through required synchronous and asynchronous paths.
- ☐ Span names and attributes are stable, bounded, and privacy reviewed.
- ☐ Instrumentation avoids duplicate spans and measures overhead.
- ☐ Sampling rates, decisions, drops, and backend ingestion are visible.
- ☐ A trace can pivot to correlated logs, service metrics, and release context.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
One request creates many trace roots	Propagation headers are lost, blocked, or replaced	Validate client, proxy, service and message context handling	One test request forms the expected connected trace
Trace volume explodes	Duplicate auto/manual instrumentation or span per loop item	Remove duplication and choose meaningful boundaries	Span rate and cost return within budget
Errors missing from samples	Head sampling discards low-frequency failures	Use error-aware or tail sampling with bounded capacity	Controlled failure traces are retained at expected rate
Trace says success, request failed	Status and error semantics are not recorded correctly	Set span status and outcome from protocol/domain behavior	Failed test request is classified consistently

Hands-on Lab 7: Trace a Request Across Two Services

Prerequisites

- Two simple services or an OpenTelemetry demo subset.
- An OpenTelemetry SDK or auto-instrumentation agent.
- A collector debug exporter or trace backend.

Implementation

1. Instrument an inbound request in Service A and an outbound request to Service B.
2. Verify trace context reaches Service B and produces a parent-child relationship.
3. Add a database or simulated dependency span and a domain span such as `reserve_inventory`.
4. Include service name, environment, version, route, status, dependency, and error type attributes.
5. Add trace and span identifiers to structured logs.
6. Create latency and failure scenarios, then compare complete traces.
7. Apply sampling and measure retained, dropped, and error-trace behavior.

Validation Evidence

- A single test request produces a connected multi-service trace.
- Logs link to the correct trace and span.
- Sampling and instrumentation overhead are measured and documented.

Cleanup

- Stop instrumentation agents, collector, services, and trace backend.
- Remove test data, local storage, temporary certificates, and synthetic identifiers.

Interview Questions and Model Answers

1. What is the difference between a trace and a span?

A trace represents an end-to-end operation; a span represents one timed operation within that trace, with context and relationships.

2. Why do traces become fragmented?

Context may not be injected, propagated, permitted through proxies, extracted, or carried across asynchronous messages.

3. What is tail sampling?

A collector retains trace state and decides after seeing more of the trace, allowing policies based on errors or latency at the cost of memory, delay, and complexity.

4. When is manual instrumentation needed?

When automatic instrumentation lacks domain operations, meaningful attributes, asynchronous relationships, or application-specific events needed for diagnosis.

Chapter Summary

- Tracing depends on reliable context propagation.
- Span design should reflect meaningful operations and bounded attributes.
- Automatic and manual instrumentation should complement rather than duplicate each other.
- Sampling affects cost, evidence quality, and rare-failure visibility.

Chapter 8: SLIs, SLOs, Error Budgets and User-Centric Measurement

Defining reliability through what users experience

Why this chapter matters: Infrastructure health can remain green while customers experience failures. Service objectives translate user expectations into measurable indicators, decision rules, and balanced reliability investment.

Learning Objectives

- Distinguish SLIs, SLOs, SLAs, error budgets, and operational targets.
- Select availability, latency, quality, freshness, durability, and correctness indicators.
- Define valid events, good events, windows, exclusions, and data sources.
- Calculate budget consumption and burn rate conceptually.
- Connect objectives to alerting, release, incident, and prioritization decisions.

Core Concepts

SLI, SLO and SLA

An SLI is a measured property of service behavior. An SLO is the target level over a defined window. An SLA is a business agreement that may include consequences. Internal objectives should support decisions even when no external agreement exists.

User-centric indicators

Useful indicators approximate what users care about: successful requests, latency below a threshold, correct results, fresh data, durable writes, or completed business workflows. Easy infrastructure measurements are supporting evidence, not substitutes.

Windows and budgets

An objective such as 99.9% good events over 30 days permits 0.1% bad events. The error budget is the allowed unreliability. Burn rate describes how quickly the budget is being consumed relative to the allowed rate.

Decision policy

Objectives should state what happens when budget is healthy, at risk, or exhausted. Responses may include investigation, release caution, reliability work, or executive risk acceptance. Error budgets should guide judgment, not become an automatic punishment mechanism.

Architecture and Operating Decisions

- Start from user journeys and consequences, then work backward to measurable indicators.
- Define numerator, denominator, thresholds, window, exclusions, source, owner, and data-quality checks.
- Use multiple windows or burn-rate views to detect fast and slow objective risk.
- Separate service objectives from monitoring-system availability and internal component targets.
- Review objectives when user expectations, architecture, traffic, or business risk changes.

Reference Configuration or Command Pattern

```
service: checkout-api
objective: successful_order_submission
window: 30d
target: 99.9
sli:
  type: request_based
  good_events: "valid order requests completed with accepted outcome within 800 ms"
  valid_events: "all non-test order requests reaching the service boundary"
  exclusions:
    - explicitly cancelled by user before submission
  source: edge request metrics
  owner: checkout-platform
decision_policy:
  budget_healthy: normal release policy
  budget_at_risk: reliability review required
  budget_exhausted: pause high-risk changes unless risk owner approves
```

Interpretation: The objective defines measurement and decision behavior. The exact target must be justified by user expectation, architecture, cost, and business risk.

Production Readiness Checklist

- ☐ Objectives are tied to named user journeys and owners.
- ☐ Good and valid events, thresholds, windows, exclusions, and sources are explicit.
- ☐ Measurement data has freshness, completeness, and correctness checks.
- ☐ Budget status influences release and reliability decisions through a documented policy.
- ☐ Objectives and targets are reviewed with product, engineering, and operations stakeholders.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
SLO is always 100%	Target was chosen aspirationally without cost or risk analysis	Set a realistic objective that enables trade-offs and improvement	Stakeholders agree to target and decision policy
Dashboard shows 99.9%, users complain	Indicator measures component uptime, not user journey	Measure at service boundary or user workflow	Controlled user failure affects the SLI correctly
Budget changes after query edit	SLI definition and query are not versioned or tested	Store specification, query, tests, and change history together	Historical and new calculations are explainable
Alerts fire on tiny blips	Single threshold ignores window and budget impact	Use objective risk and multi-window burn-rate logic	Transient event does not page; material burn does

Hands-on Lab 8: Define and Test a Service Objective

Prerequisites

- A sample service with success, failure, and latency measurements.
- At least one user journey and owner.
- A spreadsheet, query tool, or simple script for event calculation.

Implementation

1. Interview the service owner or simulate the user expectation for one critical journey.
2. Define valid events, good events, latency threshold, window, exclusions, source, and owner.
3. Choose a target and calculate the allowed bad-event proportion.
4. Create sample traffic with successful, failed, and slow events.
5. Calculate the SLI and remaining budget for the sample window.
6. Test how a fast severe incident and a slow degradation consume budget differently.
7. Write a decision policy for healthy, at-risk, and exhausted budget states.

Validation Evidence

- The SLO specification, query or calculation, and test dataset are version controlled.
- Controlled good, bad, excluded, and missing-data cases are validated.
- A stakeholder decision policy is attached to the objective.

Cleanup

- Delete synthetic request data and temporary dashboards.
- Remove test alerts and reset any modified service thresholds.

Interview Questions and Model Answers

1. What is an error budget?

It is the amount of unreliability permitted by the SLO over its measurement window and can guide the balance between feature velocity and reliability work.

2. Why not use CPU utilization as an SLI?

CPU may explain a failure but does not directly measure whether users received a correct and timely service outcome.

3. How do SLAs differ from SLOs?

An SLA is a business agreement that may include consequences; an SLO is an operational reliability target used to manage the service.

4. What makes an SLO actionable?

It has a clear user journey, measurement definition, owner, trustworthy data, target, window, and decision policy.

Chapter Summary

- Reliability should be measured through user-relevant outcomes.
- SLIs require precise event, threshold, window, and data definitions.
- Error budgets convert objectives into risk and release decisions.
- Objectives must be versioned, tested, and reviewed.

Chapter 9: Dashboards, Alerting and Diagnostic Workflow Foundations

Building views and notifications that drive correct action

Why this chapter matters: A dashboard is not useful because it contains many panels, and an alert is not useful because it detects every anomaly. Both should support a specific decision with trustworthy context and a clear next action.

Learning Objectives

- Design dashboards for service overview, investigation, capacity, release, and platform health.
- Apply hierarchy, units, thresholds, time ranges, variables, and annotations correctly.
- Distinguish records, notifications, tickets, and pages.
- Write actionable alerts with severity, ownership, runbook, and inhibition or grouping behavior.
- Use a systematic investigation workflow based on symptoms, scope, change, dependencies, and evidence.

Core Concepts

Dashboard purpose

Overview dashboards answer whether users are affected and where to investigate. Diagnostic views support deeper component analysis. Capacity and release views answer different questions and should not be mixed into one wall of panels.

Visualization integrity

Correct units, axes, aggregation, denominators, rate windows, missing-data treatment, and time alignment matter more than visual decoration. Annotations for deployment and configuration changes reduce investigation time.

Alert actionability

A page should require timely human action for material impact or objective risk. Tickets and notifications can handle nonurgent work. Alerts need owner, severity, summary, impact, evidence, runbook, and clear resolution behavior.

Investigation workflow

Begin with the user symptom and time window. Confirm scope, recent change, objective impact, service dependencies, telemetry quality, and competing hypotheses. Change one variable at a time when possible and preserve a timeline.

Architecture and Operating Decisions

- Create one primary service overview organized around user journeys and objectives.
- Use links from overview to metrics, logs, traces, runbooks, deployments, and ownership.
- Alert on actionable symptoms and objective risk; use causes as diagnostic evidence where possible.
- Group, deduplicate, inhibit, silence with expiry, and test routing and escalation.
- Review dashboards and alerts after incidents and remove those that did not help.

Reference Configuration or Command Pattern

```
groups:
- name: checkout-slo
  rules:
  - alert: CheckoutHighErrorBudgetBurn
    expr: checkout_error_budget_burn_rate_1h > 14
    for: 5m
    labels:
      severity: page
      service: checkout-api
      team: checkout-platform
    annotations:
      summary: "Checkout is consuming error budget rapidly"
      impact: "Order submissions may fail or exceed the latency objective"
      dashboard: "https://grafana.example/d/checkout-overview"
      runbook: "https://runbooks.example/checkout/high-burn"
```

Interpretation: The recording metric and thresholds are illustrative. Production burn-rate alerting should use validated SLO calculations and multiple windows appropriate to the objective.

Production Readiness Checklist

- ☐ Each dashboard has a stated audience, decision, owner, source, and review date.
- ☐ Units, aggregations, rates, thresholds, missing data, and time zones are correct.
- ☐ Pages are actionable, user-impact oriented, routed, grouped, deduplicated, and tested.
- ☐ Every urgent alert links to owner, dashboard, runbook, relevant changes, and evidence.
- ☐ Post-incident reviews evaluate whether each dashboard and alert helped.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Dashboard panels disagree	Queries use different labels, windows, sources, or denominators	Standardize definitions and show source metadata	Controlled dataset produces consistent values
Alert flaps repeatedly	Threshold, evaluation window, missing data, or resolution logic is unstable	Add duration, hysteresis, inhibition and data-quality handling	Controlled oscillation does not create repeated pages
On-call receives alert with no action	Condition is informational or owner/runbook is missing	Downgrade, ticket, enrich, or remove the alert	Responder can state the required action immediately
Dashboard is green during outage	View monitors components rather than user path or telemetry is missing	Add SLI, synthetic test and collection-health signals	Controlled outage appears as user impact and data state

Hands-on Lab 9: Build a Service Overview and Alert Review

Prerequisites

- A metric source with request and host measurements.
- Grafana or another dashboard tool.
- A sample service and one objective.

Implementation

1. Write the five questions the service overview must answer.
2. Create panels for objective status, traffic, errors, latency distribution, saturation, dependency health, and telemetry freshness.
3. Add deployment annotations and links to logs, traces, owner, and runbook.
4. Verify units, aggregation, time range, missing data, and multi-instance behavior.
5. Create one urgent objective-risk alert and one nonurgent capacity ticket condition.
6. Test routing, grouping, deduplication, silence expiry, recovery, and missing-data behavior.

7. Run a mock incident and record which panels and alerts actually supported diagnosis.

Validation Evidence

- Dashboard JSON or configuration is version controlled.
- Screenshots or exported data demonstrate normal, failure, missing-data, and recovery states.
- Alert tests prove routing, grouping, deduplication, and actionable content.

Cleanup

- Remove test alerts, silences, routes, dashboards, users, and temporary data sources.
- Restore any modified thresholds and service conditions.

Interview Questions and Model Answers

1. What makes an alert actionable?

It represents material impact or risk that requires a timely human decision and includes ownership, context, evidence, and a safe next action.

2. What should appear first on a service dashboard?

User journey and objective status, then traffic, errors, latency, saturation, dependencies, changes, and telemetry health.

3. Why add deployment annotations?

They help correlate a symptom with changes without assuming every incident was caused by the latest release.

4. How do you reduce alert fatigue?

Remove nonactionable alerts, use objective risk, improve grouping and inhibition, correct flapping and missing data, route accurately, and review alert outcomes.

Chapter Summary

- Dashboards and alerts should support explicit decisions.
- Visualization errors can be operationally dangerous.
- Urgency is based on required action and impact, not raw anomaly.
- Incident reviews should improve or retire observability objects.

Chapter 10: Foundation Observability Platform Project

Integrating service inventory, Linux telemetry, metrics, logs, traces, SLOs, dashboards, and alerts

Why this chapter matters: The foundation is complete only when a second engineer can deploy the stack, generate a known incident, observe user impact, correlate evidence, execute recovery, and verify that telemetry remains trustworthy.

Learning Objectives

- Design a small but production-informed observability architecture.
- Deploy Prometheus, Grafana, node-level metrics, a sample service, and an OpenTelemetry collection path.
- Create structured logs, traces, service metrics, an SLO, dashboard, alert, and runbook.
- Test telemetry loss, service failure, resource saturation, and recovery.
- Produce a portfolio-ready evidence package and improvement roadmap.

Core Concepts

Minimum viable platform

The project includes a sample service, host metrics, service metrics, structured logs, traces, collection, storage, visualization, objective calculation, alerting, ownership, runbooks, and platform self-monitoring.

Trustworthy validation

A successful container start is not validation. Test the user path, known telemetry, timestamps, labels, correlation, objective calculation, alert routing, recovery, and data-loss behavior.

Failure exercises

Introduce bounded failures: stopped service, dependency latency, CPU pressure, collector outage, scrape failure, and invalid configuration. Each exercise needs a hypothesis, stop condition, evidence, recovery, and post-test cleanup.

Portfolio evidence

A credible project contains architecture, repository structure, configuration, ADRs, service records, SLO, dashboard source, alert tests, runbook, incident timeline, measured results, known limitations, cost estimate, and next steps.

Architecture and Operating Decisions

- Use Docker Compose or an isolated lab cluster with explicit resource limits and cleanup.
- Keep the architecture simple enough to understand every data path.
- Pin images or versions for reproducibility and record resolved digests where possible.
- Use synthetic data and never embed real credentials or production configuration.
- Treat the platform as a service with health checks, storage, retention, backup, and recovery tasks.

Reference Configuration or Command Pattern

```

services:
  prometheus:
    image: prom/prometheus:<pinned-version>
    command: ["--config.file=/etc/prometheus/prometheus.yml"]
    volumes:
      - ./prometheus:/etc/prometheus:ro
      - prometheus-data:/prometheus
    ports: ["9090:9090"]
  grafana:
    image: grafana/grafana:<pinned-version>
    volumes:
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
      - grafana-data:/var/lib/grafana
    ports: ["3000:3000"]
  node-exporter:
    image: prom/node-exporter:<pinned-version>
    ports: ["9100:9100"]
  otel-collector:
    image: otel/opentelemetry-collector-contrib:<pinned-version>
    command: ["--config=/etc/otelcol/config.yaml"]
    volumes:
      - ./otel/config.yaml:/etc/otelcol/config.yaml:ro
    ports: ["4317:4317", "4318:4318"]
  demo-service:
    build: ./app
    environment:
      OTEL_EXPORTER_OTLP_ENDPOINT: http://otel-collector:4318
    ports: ["8080:8080"]
volumes:
  prometheus-data: {}
  grafana-data: {}

```

Interpretation: Replace placeholders with reviewed versions and verify current image names, configuration, ports, security, and licensing in official documentation. The lab should bind only to approved interfaces and use nonproduction credentials.

Production Readiness Checklist

- ☐ Architecture, owners, data flows, ports, identities, storage, retention, and failure boundaries are documented.
- ☐ Service, host, collection, storage, query, dashboard, alert, and user-path health are independently validated.
- ☐ Metrics, logs, and traces share stable service and environment context.
- ☐ SLO, dashboard, alert, and runbook are version controlled and tested.
- ☐ Failure drills, cleanup, limitations, cost, and improvement backlog are recorded.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Prometheus target is down	Wrong address, network, path, TLS, or service readiness	Inspect target error, container network and endpoint directly	Target is up and known metric changes after workload
Grafana shows no data	Data source URL or query context is incorrect	Test data source, query directly, verify time range and labels	Dashboard displays controlled metric change
Trace/log correlation fails	Service identity or context propagation differs	Normalize resource fields and trace identifiers	One request links metric exemplar or log to trace
Alert works only manually	Rule evaluation, routing, grouping, receiver or recovery is untested	Run end-to-end alert lifecycle test	Notification fires, groups, resolves, and records evidence

Hands-on Lab 10: Build the Volume 11A Observability Foundation

Prerequisites

- Docker Engine with Compose, or an approved isolated lab cluster.
- At least 8 GB RAM and sufficient temporary disk space.

- A Git repository and synthetic sample service.
- No production credentials, identifiers, or customer data.

Implementation

1. Create the repository structure for app, Prometheus, Grafana provisioning, OpenTelemetry Collector, service catalog, SLO, alerts, runbooks, tests, and evidence.
2. Deploy the sample service, Prometheus, Grafana, node exporter, and Collector with pinned versions and bounded resources.
3. Instrument service metrics, JSON logs, and a two-span trace; apply consistent service, environment, and version context.
4. Create a service record, one user-centric SLO, a service overview dashboard, and an actionable alert with runbook.
5. Generate normal, failed, slow, and saturated traffic and validate metrics, logs, traces, SLI, dashboard, and alert lifecycle.
6. Stop one collection component and one service component separately; observe data freshness, platform health, user impact, recovery, and possible data loss.
7. Publish architecture, ADRs, test results, incident timeline, cost/retention assumptions, limitations, cleanup, and the backlog for Volume 11B.

Validation Evidence

- A fresh environment can be deployed from version-controlled configuration.
- A reviewer can follow one request through service metric, structured log, and trace.
- The SLO and alert respond correctly to controlled failure and recovery.
- The final report includes platform self-monitoring and data-loss findings.

Cleanup

- Export sanitized evidence, then run docker compose down -v or the equivalent approved teardown.
- Remove images, volumes, generated data, local credentials, temporary DNS, and firewall changes not required for future study.

Interview Questions and Model Answers

1. What proves an observability platform works?

Known telemetry and user-path tests must traverse collection, storage, query, visualization, alerting, and recovery with expected identity, freshness, and data quality.

2. Why test telemetry failure separately from service failure?

A service outage and an observation outage create different risks. Without separate testing, missing data may be mistaken for healthy behavior.

3. What should be included in a portfolio observability project?

Architecture, code, service catalog, instrumentation, objectives, dashboards, alerts, runbooks, failure drills, evidence, trade-offs, cost, security, and limitations.

4. What is the next technical depth after this foundation?

Prometheus architecture, exporters, service discovery, PromQL, recording rules, Alertmanager, scaling, high availability, federation, and long-term metrics operations.

Chapter Summary

- The capstone integrates all Volume 11A foundations.
- End-to-end validation matters more than successful startup.
- The observability system must expose its own health and data quality.
- The completed project becomes the base for advanced Prometheus engineering in Volume 11B.

Part A Command and Configuration Reference

Use bounded durations, approved privilege, and safe environments. Record timestamps and explain what each command proves.

Area	Commands or artifacts	Purpose
Host overview	date -ls; uptime; uname -a; lscpu; free -h	Time, load, kernel, CPU topology and memory context
Processes	ps; top; pidstat; /proc/<pid>; systemctl; journalctl	Process state, resource use, limits, service status and logs
CPU/memory pressure	vmstat; mpstat; /proc/pressure/*; dmesg	Run queue, CPU states, reclaim, pressure and kernel events
Storage	df -hT; df -ih; lsblk; iostat -xz; mount	Capacity, inodes, topology, latency, queues and mounts
Network	ip; ss; ping; tracepath; dig; curl; openssl s_client	Interface, socket, path, DNS, HTTP and TLS evidence
Metrics	/metrics endpoint; Prometheus target page; query expression	Metric exposition, collection state and time-series analysis
Logs	JSON schema; journalctl; collector self-telemetry	Event context, parsing, backlog, drops and routing
Traces	traceparent; OTLP endpoint; collector pipeline	Context propagation and span export
Objectives	SLI specification; event query; budget calculation	User-centric reliability and decision policy
Platform	service catalog; dashboard JSON; alert rules; runbook	Ownership, visualization, action and recovery evidence

Observability Foundation Completion Checklist

- ☐ I can explain monitoring, observability, telemetry, evidence, diagnostics, audit, and SRE without using product names.
- ☐ I can map a user journey to components, dependencies, owners, metrics, logs, traces, SLOs, dashboards, alerts, and runbooks.
- ☐ I can interpret Linux CPU, load, memory, I/O, filesystem, network, process, service, and kernel evidence.
- ☐ I can choose counters, gauges, histograms, labels, structured log fields, span boundaries, and context attributes safely.
- ☐ I can identify cardinality, missing data, duplicate data, delayed data, parser failure, sampling, and collection backpressure.
- ☐ I can write a user-centric SLI and SLO with valid events, good events, target, window, exclusions, source, owner, and decision policy.
- ☐ I can build a service overview dashboard and an actionable alert with correct routing, grouping, runbook, and recovery behavior.
- ☐ I can deploy, break, recover, validate, document, and clean up the Volume 11A foundation project.

Ten Additional Practice Projects

1. Host Baseline Reporter

Create a script that captures bounded Linux host evidence with timestamps and produces a sanitized Markdown incident snapshot.

2. Telemetry Contract Validator

Validate service names, environment labels, metric types, log fields, trace context, and forbidden sensitive attributes in CI.

3. Cardinality Budget Test

Generate bounded and unbounded label patterns, measure series growth, and create a release gate for expected cardinality.

4. Structured Log Quality Gate

Test JSON schema, severity, event names, trace correlation, secret leakage, and maximum record size.

5. Trace Propagation Matrix

Verify context across HTTP, background jobs, and one messaging boundary with success and failure cases.

6. SLO Calculator

Calculate good events, valid events, objective attainment, remaining budget, and fast/slow burn from a synthetic dataset.

7. Dashboard Review Linter

Create a checklist or script for owner, purpose, units, links, missing data, time range, annotations, and review date.

8. Alert Lifecycle Test

Automate firing, grouping, deduplication, silence expiry, escalation, resolution, and evidence capture in a lab receiver.

9. Observability Failure Game Day

Stop collectors, storage, DNS, and service components separately; document visibility, data loss, recovery, and false-green risks.

10. Service Onboarding Package

Create a catalog record, instrumentation checklist, SLO, dashboard, alert, runbook, and decommission workflow for a new service.

30-Day Foundation Practice Plan

Days	Focus	Required output
1-3	Operating model and questions	Service charter, owner map and user-journey questions
4-6	Telemetry signals and Collector	Three-signal pipeline and failure test
7-9	Service catalog and contract	Schema, three service records and drift check
10-13	Linux monitoring	Host baseline and controlled CPU/I/O/service incident
14-17	Metrics	Metric contract, histogram, reset and cardinality tests
18-20	Logs	Structured schemas, correlation and synthetic-secret test
21-23	Tracing	Two-service trace, logs correlation and sampling notes
24-26	SLOs	Objective specification, dataset and budget decision policy
27-28	Dashboards and alerts	Service overview and tested alert lifecycle

Days	Focus	Required output
29-30	Capstone	Integrated platform, game day, report and cleanup evidence

Glossary

Term	Definition
Alert	An active condition produced by an alerting rule; a notification is one possible delivery of that alert.
Cardinality	The number of unique time series or distinct field combinations produced by dimensions.
Context propagation	Transfer of trace and related context across process or service boundaries.
Counter	A cumulative metric that increases except when reset.
Error budget	The unreliability permitted by an SLO over its window.
Gauge	A metric representing a value that can increase or decrease.
Histogram	A metric that counts observations in buckets and records sum and count.
Log record	A timestamped event record with severity, body, resource context, and attributes.
Observability	The capability to infer internal system behavior from externally available outputs.
OTLP	OpenTelemetry Protocol used to transport telemetry between compatible components.
Resource attributes	Context describing the entity producing telemetry, such as service, environment, host, or version.
SLI	Service Level Indicator: a quantitative measure of service behavior.
SLO	Service Level Objective: a target for an SLI over a defined window.
Span	A timed operation within a distributed trace.
Telemetry	System outputs such as metrics, logs, traces, profiles, and events.
Time series	Timestamped samples identified by a metric name and dimensions.
Trace	An end-to-end operation represented by related spans.

Official Documentation Index

Product behavior, versions, feature status, default ports, configuration syntax, licensing, and operational guidance change. Verify current official documentation before production use.

- [OpenTelemetry Documentation](#)
- [OpenTelemetry Signals](#)
- [OpenTelemetry Collector](#)
- [OpenTelemetry Collector Configuration](#)

- [Prometheus Overview](#)
- [Prometheus Getting Started](#)
- [Prometheus Data Model](#)
- [Prometheus Alerting Practices](#)
- [Grafana Documentation](#)
- [Grafana Getting Started](#)
- [Google SRE Book - Service Level Objectives](#)
- [Google SRE Book - Practical Alerting](#)
- [Linux Kernel Pressure Stall Information](#)

Volume 11A completion: the learner now has the conceptual, Linux, telemetry, SLO, dashboard, alerting, and platform-validation foundation required for Volume 11B - Prometheus Engineering and PromQL.